Notes on Liu-Layland bounds for RM scheduling

Luca Seggiani
2026

1 Rate Monotonic scheduling bounds

This note examines the derivation of the Liu–Layland utilization bound for **RM** (*Rate Monotonic*) scheduling, starting with the two-task case and extending the argument to an arbitrary number of tasks. The goal is to understand how task periods and execution times constrain the maximum processor utilization that can be guaranteed schedulable under a fixed-priority assignment. The derivation follows the original 1973 paper, making explicit the critical-instant argument, the minimization of the utilization bound, and the assumptions required at each step. Finally, the restriction on period ratios is removed, establishing the general bound

$$U_{\text{bound}} = m\left(2^{1/m} - 1\right).$$

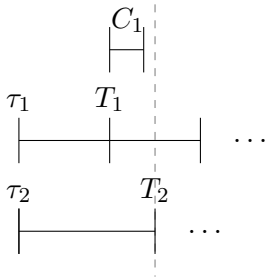
The original article, for reference, is *Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment* (1973), by C. L. Liu and James W. Layland.

1.1 Two-process case

Let's first consider two processes, τ_1 and τ_2, with $T_1 < T_2$ (and thus τ_1 at higher priority than τ_2). We seek to evaluate the maximum CPU utilization U, that it is the maximum service time the two processes can request.

Let's fix C_1 and leave C_2 free. We consider the critical zone of τ_2, that is a period T_2 over which τ_2 executes interleaved with τ_1. To see the maximum value of C_2 (from which we'll derive the maximum utilization U) we have to consider two distinct situations:

1. τ_1 is able to finish servicing its last request within the critical zone of τ_2, before the deadline T_2 of τ_2 arrives.



This is expressed by the inequality:

$$C_1 \leq T_2 - T_1 \left\lfloor \frac{T_2}{T_1} \right\rfloor$$

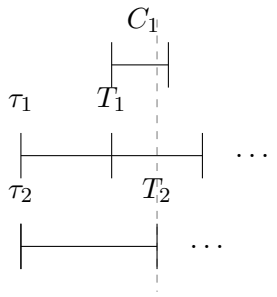
We can calculate the maximum value of C_2 by taking the whole T_2 period and assuming the τ_1 process is fine taking its time C_1 every time a request for it appears (that is, $\left\lceil \frac{T_2}{T_1} \right\rceil$ for a period of T_1) in T_2:

$$C'_{2\text{max}} = T_2 - C_1 \left\lceil \frac{T_2}{T_1} \right\rceil$$

From this we compute the maximum utilization U'_{max}:

$$U'_{\text{max}} = 1 + C_1 \left(\frac{1}{T_1} - \frac{1}{T_2} \left\lceil \frac{T_2}{T_1} \right\rceil \right)$$

2. τ_1 is not able to finish servicing its last request within the critical zone of τ_2, before the deadline T_2 of τ_2 arrives. This means that its C_1 execution time, issued at the last request within T_2, "bleeds" after the T_2 deadline.



This is expressed by the opposite of the inequality above:

$$C_1 \geq T_2 - T_1 \left\lfloor \frac{T_2}{T_1} \right\rfloor$$

We can calculate the maximum value of C_2 differently than above. The idea is that, for each request apart from the last one, τ_1 will execute in C_1 within a T_1 period, leaving $T_1 - C_1$ time free for τ_2. The last period that falls in T_2, as we said, is not going to be enough to allow τ_1 to finish, and as its priority is higher than τ_2 it's going to "steal away" all of the end of τ_2's period. Mathematically:

$$C_2''_{\text{max}} = \left\lfloor \frac{T_2}{T_1} \right\rfloor (T_1 - C_1)$$

Again, from this we compute the maximum utilization U''_{max}:

$$U''_{\text{max}} = \frac{T_1}{T_2} \left\lfloor \frac{T_2}{T_1} \right\rfloor + C_1 \left(\frac{1}{T_1} - \frac{1}{T_2} \left\lfloor \frac{T_2}{T_1} \right\rfloor \right)$$

The two equations we got for U'_{max} and U''_{max} are respectively decreasing and increasing in C_1, that is the least maximum bound is given at the intersection:

$$C_1 = T_2 - T_1 \left\lfloor \frac{T_2}{T_1} \right\rfloor$$

Now the algebra gets fucked up fast so we first say:

$$q = \frac{T_2}{T_1}$$

And subsequently:

$$\lfloor q \rfloor = f, \quad \lceil q \rceil = c$$

Now we compute the two bounds separately:

1. For $U'_{\max}$ we have:

$$U'_{\max} = 1 + \left(T_2 - T_1 \left\lfloor \frac{T_2}{T_1} \right\rfloor\right)\left(\frac{1}{T_1} - \frac{1}{T_2}\left\lceil \frac{T_2}{T_1} \right\rceil\right) = 1 + q - \lceil q \rceil - \lfloor q \rfloor + \frac{1}{q}\lfloor q \rfloor \lceil q \rceil$$
$$= 1 + q - c - f + \frac{1}{q}cf$$

2. For $U'_{\max}$ we have:

$$U''_{\max} = \frac{T_1}{T_2}\left\lfloor \frac{T_2}{T_1} \right\rfloor + \left(T_2 - T_1 \left\lfloor \frac{T_2}{T_1} \right\rfloor\right)\left(\frac{1}{T_1} - \frac{1}{T_2}\left\lfloor \frac{T_2}{T_1} \right\rfloor\right) = \frac{\lfloor q \rfloor}{q}(1 + \lfloor q \rfloor) + q - 2\lfloor q \rfloor$$
$$= \frac{f}{q}(1 + f) + q - 2f$$

Ain't that something. At this point we first need to prove that $U'_{\max}$ and $U''_{\max}$ are in fact equal. Dealing with ceiling and floor functions is tricky so we split the case where $q \in \mathbb{Z}^+$ and $q \in \mathbb{Q}^+$ (we don't really care about negative numbers as these are time periods, and we need not evaluate all of $\mathbb{R}$ as these are ratios between integers, though I imagine in $\mathbb{R}$ the calculations are similar).

- In $\mathbb{Z}^+$ we can say $q = f = c$ and the equations simplify quickly to 1:

$$U'_{\max} = 1 + q - q + \frac{1}{q}q^2 = 1$$
$$U'_{\max} = \frac{q}{q}(1 + q) + q - 2q = 1$$

- In $\mathbb{Q}^+$ we can similarly say that $c = f + 1$, which leads to:

$$U'_{\max} = 1 + q - f - 1 - f + \frac{1}{q}(f(f + 1)) = q - 2f + \frac{f}{q}(1 + f)$$
$$U'_{\max} = q - 2f + \frac{f}{q}(1 + f)$$

Now that we have a function $U_{\max}$ we have to minimize it to find the least maximum bound. A few considerations:

- $q \geq 1$, because:

$$q = \frac{T_2}{T_1} \geq 1 \implies T_2 > T_1$$

  Had it been otherwise τ_1 and τ_2 would swap priorities;
- As we found out before, $U_{\max} = 1$ for each $q \in \mathbb{Z}^+$;
- For each point $q \in \{\mathbb{Q}^+ \setminus \mathbb{Z}^+\}$, f is basically fixed, and q is our free variable. We can ignore the discontinuities at $q \in \mathbb{Z}^+$ as we are going to find the lower bound is lower than 1.

  This being said, we derive considering f as a parameter:

$$\frac{d}{dq}\left(q - 2f + \frac{f}{q}(1 + f)\right) = 1 - \frac{f(1 + f)}{q^2}$$

  The minimum is found by setting this to zero:

$$1 - \frac{f(1 + f)}{q^2} = 0 \implies q^* = \sqrt{f(1 + f)}$$

Let's think about what this means. Our function has a stationary point q_i for each point which respects:

$$q_i = \sqrt{f_i(1+f_i)}$$

With $f_i = 1, 2, ...$ as q_1 with $f_i = 1$ comes out to $\sqrt{2}$, just above our $q > 1$ bound. By quick evaluation at the first few points we see that they are indeed below 1, so all we really have to do is find out which one of these is the lowest.

By substituting back in the original expression we get:

$$U^*_{\text{max}} = \sqrt{f(1+f)} - 2f + \frac{f}{\sqrt{f(1+f)}}(1+f) = 2\left(\sqrt{f(1+f)} - f\right)$$

Let's derive, this time with f as the variable:

$$\frac{d}{df}\left(2\left(\sqrt{f(1+f)} - f\right)\right) = 2\left(\frac{1+2f}{2\sqrt{f(1+f)}} - 1\right)$$

Now, if only this derivative was greater than 0, we could say that the sequence q_i given by $f_i = 1, 2, 3...$ is non decreasing and thus the minimum is found at q_1 with $f_i = 1$. Luck be on our side, we get:

$$\begin{aligned} 2\left(\frac{1+2f}{2\sqrt{f(1+f)}} - 1\right) &> 0 \\ \frac{1+2f}{2\sqrt{f(1+f)}} &> \frac{1}{2} \\ 1+2f &> 1\sqrt{f(1+f)} \\ 1+3f^2+3f &> 0 \end{aligned}$$

Which is definitely true for $f > 0$. We are happy.

In conclusion, what we got is that our CPU utilization function U_{max} has a least maximum bound in:

$$q^* = \frac{T_2}{T_1} = \sqrt{2}$$

Which we got by imposing $f_i = 1$ in the $q_i = \sqrt{f_i(1+f_i)}$ expression. At this point all that's left is evauating the least maximum utilization:

$$U^*_{\text{max}} = \sqrt{2} - 2 + \frac{1}{\sqrt{2}} \cdot 2 = 2\sqrt{2} - 2 = 2(2^{\frac{1}{2}} - 1)$$

1.2 Multi-process case

This should be nothing more than a generalization of the above case. The original article is followable so I'll stay concise.

First off we take a set of m processes, ordered as usual with periods:

$$T_1 < T_2 < ... < T_m$$

So that the priorities are ordered in the opposite direction, with T_1 having the highest priority and T_m the lowest priority. **Note** that for now we make an assumption: the ratio between any two request periods must be less than 2.

We wish to show that if $V_1, ..., V_m$ make a minimal bound of the CPU utilization U, it must follow that:

$$C_i = T_{i+1} - T_i, \quad i \neq m$$

We show this by proving the case for C_i, and letting the rest stem from recursion. The equality to prove thus becomes:

$$C_1 = T_2 - T_1$$

Two situations to consider:

- If we perturb C_1 negatively, that is say that there exists some $\Delta > 0$ such that:

$$C_1 = T_2 - T_1 + \Delta$$

Then we can easily build a new set of runtimes C_i which fully utilize the processor:

$$\begin{cases} C'_1 & = T_2 - T_1 \\ C'_2 & = C_2 + \Delta \\ C'_3 & = C_3 \\ & \vdots \\ C'_m & = C_m \end{cases}$$

Let's consider the differences in utilization between the two sets. Only the first two elements change, so:

$$U - U' = \frac{\Delta}{T_1} - \frac{\Delta}{T_2}$$

Since $T_1 < T_2$, $\frac{1}{T_1} > \frac{1}{T_2}$, and $U > U'$. This is the first contradiction.

- If we perturb C_1 positively, that is say that there exists some $\Delta > 0$ such that:

$$C_1 = T_2 - T_1 - \Delta$$

Then we can easily build a new set of runtimes C_i which fully utilize the processor:

$$\begin{cases} C''_1 & = T_2 - T_1 \\ C''_2 & = C_2 - 2\Delta \\ C''_3 & = C_3 \\ & \vdots \\ C''_m & = C_m \end{cases}$$

Let's consider the differences in utilization between the two sets. Only the first two elements change, so:

$$U - U'' = -\frac{\Delta}{T_1} + 2\frac{\Delta}{T_2}$$

Which also leads to $U > U''$. This is the second contradiction.

It follows that indeed:

$$C_1 = T_2 - T_1$$

And by recursion:

$$C_i = T_{i+1} - T_i, \quad i \neq m$$

We can express this more compactly in a number of ways, of varying usefulness.

- First off, we are interested in C_m. By the assumption made above (the ratio between any two request periods must be less than 2) we can say that:

$$C_m = T_m - 2\sum_{i}^{m-1} C_i$$

At this point note that the sum $\sum_i^{m-1} C_i$ telescopes, so that this reduces to:

$$C_i = T_{i+1} - T_i, \quad C_m = 2T_1 - T_m$$

- "To simplify the notation" the paper now gives:

$$g_i = \frac{T_m - T_i}{T_i}$$

We can use this to give alternative definitions of C_i and C_m:

$$C_i = g_i T_i - g_{i+1} T_{i+1}, \quad C_m = T_m - 2g_1 T_1$$

From the last set of equations we can derive a workable expression for the utilization U:

$$\begin{aligned} U &= \sum_{i=1}^{m-1} \left[g_i - g_{i+1} \left(\frac{T_{i+1}}{T_i} \right) \right] + 1 - 2g_1 \left(\frac{T_1}{T_m} \right) \\ &= \sum_{i=1}^{m-1} \left[g_i - g_{i+1} \frac{g_i + 1}{g_{i+1} + 1} \right] + 1 - 2\frac{g_1}{g_m + 1} \\ &= 1 + g_m \left(\frac{g_1 - 1}{g_m + 1} \right) + \sum_{i=2}^{m-1} g_i \left(\frac{g_i - g_{i-1}}{g_i + 1} \right) \end{aligned}$$

As before, we can find a minimum by deriving with respect to g_j:

$$\frac{dU}{dg_j} = \frac{g_j^2 + 2g_j - g_{j-1}}{(g_j + 1)^2} - \frac{g_{j+1}}{g_{j+1} + 1} = 0$$

Which can be proven to give:

$$g_j = 2^{\frac{m-j}{m}} - 1, \quad j = 0, 1, 2, ..., m-1$$

At this point we only need to put the g_js back in the original equation, getting:

$$U = m(2^{\frac{1}{m}} - 1)$$

Most of the algebra wasn't notated here because I imagine it is unfathomably boring.

We still have to lift that last restriction, that the ratio between any two request periods must be less than 2. This was useful to even get an equation for C_m in the first place.

The is that any task set with a period ratio of at least 2 can be transformed into another fully utilizing task set with no greater utilization.